



Máster en  
Inteligencia  
Artificial  
**UMU**

EXTENSIONES DE MACHINE  
LEARNING

# **INFORME 1: Aprendizaje por Refuerzo**

*PASTOR BELTRÁ, MIGUEL*



# ÍNDICE

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| <b>1. Introducción.....</b>                                     | <b>3</b>  |
| <b>2. Desarrollo.....</b>                                       | <b>4</b>  |
| <b>3. Algoritmos.....</b>                                       | <b>5</b>  |
| 3.1. Parte 1: Problema de los k-brazos.....                     | 5         |
| 3.2. Parte 2: Métodos Tabulares (Entornos Complejos).....       | 5         |
| 3.3. Parte 3: Métodos Aproximados (Deep RL).....                | 6         |
| <b>4. Evaluación.....</b>                                       | <b>7</b>  |
| 4.1. Parte 1: El problema de los k-brazos.....                  | 7         |
| 4.2. Parte 2: Entornos Complejos (Métodos Tabulares).....       | 7         |
| 4.3. Parte 3: Métodos Aproximados (Deep RL en CartPole-v1)..... | 8         |
| <b>5. Conclusiones.....</b>                                     | <b>9</b>  |
| <b>6. Uso de la IA generativa.....</b>                          | <b>10</b> |
| <b>7. Bibliografía y referencias.....</b>                       | <b>11</b> |

# 1. Introducción

En esta memoria exponemos el trabajo realizado sobre **Aprendizaje por Refuerzo (RL)**, un campo de la Inteligencia Artificial que se centra en cómo un agente puede aprender a tomar las mejores decisiones posibles dentro de un entorno que no conoce y que cambia constantemente. Para ver cómo funciona en la práctica, hemos organizado la entrega en tres bloques :

- En la primera parte, estudiamos el problema del bandido de **k-brazos**. Aquí nos centramos en resolver el famoso dilema de la **exploración-explotación**: decidir cuándo nos conviene seguir usando lo que ya sabemos que funciona y cuándo toca arriesgarse a probar alternativas nuevas. Para ver cómo se comportan algoritmos como **epsilon-greedy**, **Softmax** o **UCB1**, montamos experimentos utilizando tres tipos de distribución (**Bernoulli**, **Binomial** y **Normal**), analizando cómo les afecta el ruido o los cambios de escala.
- En la segunda parte, estudiamos los entornos más complejos usando **Procesos de Decisión de Markov (MDP)** con la librería *Gymnasium*. En este caso, lo que hace el agente sí que cambia el estado del entorno de cara al futuro. Para resolverlo, implementamos y comparamos métodos tabulares: por un lado, **Monte Carlo** (basado en episodios completos) y, por otro, métodos de **Diferencias Temporales** en un solo paso, concretamente **SARSA** y **Q-Learning**. Todo esto lo ponemos a prueba en los entornos *FrozenLake-v1* y *CliffWalking-v1* para ver qué algoritmo es más rápido o cuál asume menos riesgos.
- En la tercera parte, dejamos atrás los **métodos tabulares** porque se quedan cortos cuando los escenarios tienen infinitos estados continuos. Para solucionarlo, usamos redes neuronales profundas como aproximadores de funciones. Implementamos las versiones avanzadas de **SARSA Semi-Gradiente** y **Deep Q-Networks (DQN)** en el entorno *CartPole-v1*. Aquí nuestro objetivo es analizar cómo herramientas como el *Replay Buffer* y las *Target Networks* ayudan a que la red no se vuelva inestable ni se olvide de lo que ya había aprendido.

El objetivo de este informe consiste en elaborar un estudio comparativo de los diferentes métodos a través de las métricas obtenidas.

## 2. Desarrollo

Hemos estructurado el trabajo de la siguiente forma:

- **Bandido k-brazos:** hemos modelado un entorno estacionario compuesto por una máquina de k-brazos, donde cada acción desencadena una recompensa regida por una distribución de probabilidad inicialmente desconocida para el agente. El éxito de nuestra intervención se mide a través del "arrepentimiento" (regret), una métrica que cuantifica la pérdida acumulada por no haber seleccionado siempre la acción óptima:

$$R(T) = T\mu^* - E \left[ \sum_{t=1}^T \mu_{a_t} \right]$$

Para abordar este problema, hemos seleccionado cuatro enfoques, como son los métodos epsilon-greedy, epsilon-decaimiento, UCB1 y Softmax. Hemos diseñado experimentos que ponen a prueba la sensibilidad de estos algoritmos ante cambios en el número de ensayos o en el ruido del entorno, garantizando siempre la reproducibilidad mediante el uso de semillas fijas en todas las ejecuciones.

- **Métodos tabulares:** implementamos soluciones donde almacenamos el conocimiento en tablas  $Q(s, a)$ . Aquí, nos centramos en comparar la eficiencia de los métodos de Monte Carlo frente a las Diferencias Temporales (TD). Hemos analizado cómo la actualización paso a paso nos permite aprender sin necesidad de finalizar episodios, optimizando la convergencia en entornos con recompensas dispersas.
- **Métodos aproximados:** para abordar entornos con estados infinitos (continuos), hemos sustituido las tablas por Redes Neuronales. En esta fase, nos enfrentamos a desafíos de inestabilidad y correlación de datos, los cuales logramos mitigar mediante el uso de Replay Buffers y Target Networks, permitiendo que nuestros

## 3. Algoritmos

En esta sección detallamos la lógica y las reglas que hemos seguido para implementar cada algoritmo en los notebooks.

### 3.1. Parte 1: Problema de los k-brazos

Para controlar el balance entre explorar y explotar en un entorno estático con 10 brazos, seguimos cuatro estrategias de selección de acciones:

- **epsilon-greedy:** El agente elige la mejor opción conocida (explotación) con una probabilidad de  $1 - \epsilon$ . Con una probabilidad  $\epsilon$ , elige un brazo completamente al azar (exploración). Actualizamos el valor estimado de cada brazo ( $Q(a)$ ) de forma incremental usando la media de las recompensas recibidas.
- **epsilon-decay:** Es una evolución del anterior. Empezamos con un  $\epsilon$  alto (mucho exploración al principio) que va disminuyendo en cada paso multiplicándose por un factor de decaimiento (por ejemplo, 0.99). Así, a medida que el agente acumula experiencia, se vuelve más agresivo y se dedica casi exclusivamente a explotar el mejor brazo.
- **Softmax:** En lugar de elegir al azar de forma plana, asignamos a cada brazo una probabilidad de ser elegido basada en la distribución de Boltzmann. Usamos un parámetro de temperatura ( $\tau$ ). Si  $\tau$  es alta, todos los brazos tienen probabilidades parecidas; si  $\tau$  es baja, los brazos con mejores recompensas acumuladas destacan muchísimo sobre los demás.
- **UCB1 (Upper Confidence Bound):** Este método no usa el azar. Introduce el concepto de optimismo ante la incertidumbre. Elige el brazo que maximice la suma de su recompensa estimada más un término de confianza que depende de cuántas veces se ha probado ese brazo en comparación con el total de pasos. Si un brazo se ha usado poco, su incertidumbre es alta y el algoritmo lo prioriza para descubrir si realmente es bueno o no.

### 3.2. Parte 2: Métodos Tabulares (Entornos Complejos)

Al pasar a entornos con estados cambiantes como *FrozenLake* y *CliffWalking*, creamos tablas  $Q$  (matrices de tamaño estados  $\times$  acciones) inicializadas a cero. Implementamos tres algoritmos para actualizar los valores de estas tablas:

- **Monte Carlo:** Este método espera a que el agente termine un episodio completo (llegue a la meta o muera) para aprender. Guardamos toda la trayectoria y, al final, recorremos el histórico calculando el retorno real acumulado desde cada estado visitado, actualizando la tabla  $Q$  en consecuencia.



- **SARSA (State-Action-Reward-State-Action):** Es un algoritmo *on-policy*. El agente está en un estado, elige una acción (usando epsilon-greedy), observa la recompensa y el siguiente estado, y ya elige cuál será su siguiente acción antes de actualizar. La actualización de la tabla Q se hace usando el valor de esa siguiente acción real que va a tomar.
- **Q-Learning:** Es un algoritmo *off-policy*. A diferencia de SARSA, cuando el agente transiciona al siguiente estado, estima su actualización asumiendo de forma optimista que allí tomará la acción que tenga el valor Q más alto absoluto, independientemente de que luego la política epsilon-greedy le haga explorar y tomar otra distinta.

### 3.3. Parte 3: Métodos Aproximados (Deep RL)

Para el entorno continuo de *CartPole*, sustituimos la tabla Q por una red neuronal (creada con PyTorch) que recibe como entrada el vector de 4 estados continuos y devuelve el valor Q para las 2 acciones posibles.

- **SARSA Semi-Gradiente:** Mantiene la filosofía de SARSA, pero en lugar de actualizar una celda de una matriz, calcula el error de diferencia temporal y realiza un paso de descenso de gradiente para ajustar los pesos de la red neuronal.
- **DQN (Deep Q-Network):** Implementa la lógica de Q-learning junto con dos técnicas fundamentales que añadimos para que la red no se vuelva loca entrenando:
  - **Replay Buffer:** Una cola donde guardamos las transiciones en tuplas (estado, acción, recompensa, siguiente estado, terminado). En cada paso de entrenamiento, extraemos un lote (*batch*) aleatorio de esta memoria. Esto rompe la correlación temporal de los datos y hace que el aprendizaje se parezca más a un aprendizaje supervisado estable.
  - **Target Network:** Usamos dos redes neuronales idénticas. La red principal se actualiza en cada paso, mientras que la red objetivo permanece congelada y solo se usa para calcular el valor Q del siguiente estado. Cada cierto número fijo de pasos, copiamos los pesos de la red principal a la red objetivo. Esto evita que los valores objetivo cambien constantemente en mitad del entrenamiento, solucionando el problema de "perseguir un objetivo en movimiento".

## 4. Evaluación

En esta sección analizamos cómo se han comportado los algoritmos en cada uno de los escenarios que hemos probado, comparando su rendimiento con los datos y las métricas que nos han devuelto los experimentos.

### 4.1. Parte 1: El problema de los k-brazos

Al probar las diferentes estrategias en las tres distribuciones de recompensa (Bernoulli, Binomial y Normal), observamos lo siguiente:

- **Bernoulli:** Las recompensas son binarias (0 o 1). Aquí, los métodos basados en epsilon (epsilon-greedy y epsilon-decay) funcionaron bastante bien, pero se nota que epsilon-decay saca ventaja en los pasos finales porque reduce la exploración residual y se estabiliza explotando el brazo óptimo.
- **Binomial (n=10):** Al tener más información por cada intento, UCB1 fue, sin duda, el mejor algoritmo. Logró un arrepentimiento (*regret*) casi nulo y rozó el 100% de precisión muy rápido. Como los intervalos de confianza se estrechan enseguida gracias a la calidad de la recompensa, el agente deja de explorar a ciegas casi desde el principio.
- **Softmax:** Nos dimos cuenta de que Softmax sufrió muchísimo cuando cambiamos las escalas de las recompensas. Al usar una temperatura fija de  $\tau = 1.0$ , si la diferencia numérica entre las recompensas de los brazos es muy grande (como en la Normal o la Binomial), la distribución de Boltzmann se satura o pierde el norte, haciendo que el agente explore de más y obtenga una precisión muy pobre (de apenas el 45% en algunos casos).

### 4.2. Parte 2: Entornos Complejos (Métodos Tabulares)

Al pasar a los mapas de Gymnasium, el contraste entre algoritmos *on-policy* y *off-policy*, y el impacto de la estocasticidad (el azar del entorno), nos dejaron conclusiones muy claras:

- **FrozenLake-v1:** En su versión determinista, los tres algoritmos (Monte Carlo, SARSA y Q-Learning) llegan a encontrar el camino óptimo. Sin embargo, en la versión estocástica (donde el suelo resbala y te mueves a donde no quieres), Monte Carlo sufre demasiado. Al depender de terminar todo el episodio para aprender, la aleatoriedad hace que los trayectos se vuelvan caóticos y la convergencia se ralentice una barbaridad. En cambio, SARSA y Q-Learning, al actualizar paso a paso (Diferencias Temporales), se adaptan mucho mejor al suelo resbaladizo.



- **CliffWalking-v1:** Este mapa fue perfecto para ver la diferencia real entre SARSA y Q-Learning. Como Q-Learning es *off-policy* y asume que siempre va a tomar la acción óptima, aprende el camino que pasa justo por el borde del acantilado (el más corto). El problema es que, como sigue explorando con  $\epsilon$ -greedy, esa pequeña exploración hace que se caiga muchísimo y reciba penalizaciones de -100 continuamente. SARSA, al ser *on-policy*, "sabe" que va a explorar y que puede meter la pata, así que aprende una ruta mucho más conservadora y segura, alejándose del borde para evitar accidentes.

### 4.3. Parte 3: Métodos Aproximados (Deep RL en CartPole-v1)

El entrenamiento de métodos aproximados nos dejó las siguientes conclusiones:

- **SARSA Semi-Gradiente:** Este algoritmo tiende a ser muy inestable. Al actualizar los pesos de la red usando directamente la experiencia paso a paso (*on-policy*), los datos que le llegan a la red están fuertemente correlacionados en el tiempo (lo que hace el agente ahora depende por completo de lo que acaba de hacer hace un milisegundo). Esto provocaba que el entrenamiento fuera muy irregular y que a la red le costara consolidar un aprendizaje firme, ya que las actualizaciones consecutivas rompían lo aprendido en los pasos anteriores.
- **DQN (Deep Q-Network):** El rendimiento fue muchísimo más sólido y pudimos alcanzar la puntuación máxima de 500 puntos de forma consistente. La clave fue el Replay Buffer, ya que al extraer lotes aleatorios de la memoria rompimos esa correlación temporal de la que pecaba SARSA, haciendo que el entrenamiento fuera mucho más suave y parecido a un aprendizaje supervisado clásico. Además, gracias a la Target Network, evitamos el problema de perseguir un objetivo en movimiento (que la red intente aprender valores que ella misma está cambiando a la vez), logrando una convergencia muy limpia. Eso sí, detectamos que cuando el agente se volvía perfecto y el buffer se llenaba solo con experiencias de éxito (el poste recto), la red se olvidaba de cómo reaccionar ante ángulos extremos si el poste se torcía de golpe, algo que tuvimos que controlar manteniendo una pequeña exploración residual ( $\epsilon$ ).





## 5. Conclusiones

Con esta práctica hemos comprobado cómo cambia el rendimiento de un agente según el entorno y la estrategia elegida. En los escenarios estáticos, algoritmos como UCB1 demostraron una eficiencia enorme al gestionar la incertidumbre sin perder el tiempo explorando a ciegas, mientras que en los métodos tabulares complejos vimos la importancia de adaptar la política al riesgo, donde la prudencia on-policy de SARSA evitó las caídas del temerario Q-Learning en el acantilado.

Por otro lado, el salto a los espacios continuos con redes neuronales nos dejó claro que no basta con aplicar Deep Learning directamente para solucionar un problema. Las inestabilidades sufridas por SARSA Semi-Gradiente y el posterior éxito de DQN nos enseñaron que herramientas de estabilización como el Replay Buffer y las Target Networks son obligatorias si queremos romper las correlaciones temporales y lograr un aprendizaje sólido.

## 6. Uso de la IA generativa

### **Prompt 1. ¿Cómo puedo evitar que el algoritmo epsilon-decaimiento sea menos agresivo para mis 1000 pasos?**

Para que el epsilon-decay sea menos agresivo en tus 1000 pasos, puedes reducir la tasa de decaimiento (decay) para que epsilon disminuya más lentamente, aumentar el valor mínimo de epsilon (epsilon\_min) para mantener siempre algo de exploración, calcular un decay específico que lleve a epsilon al valor deseado justo al final de los 1000 pasos, o usar un decaimiento lineal en lugar de exponencial, restando una cantidad fija por paso para tener un control más directo sobre la exploración a lo largo del entrenamiento.

### **Prompt 2. ¿Qué valores de varianza debería probar en un problema de k-brazos siguiendo una distribución normal? ¿Por qué?**

Para tu problema de k-brazos, te recomendaría probar varianzas como 1 y 20, ya que 1 representa un nivel bajo de ruido que permite ver claramente cuál brazo es mejor, mientras que 20 introduce bastante variabilidad, útil para evaluar cómo tu estrategia maneja recompensas más ruidosas y difíciles de diferenciar. Esto te da un buen rango para experimentar entre entornos más estables y más desafiantes.

### **Prompt 3. ¿Por qué Monte Carlo muestra valores de recompensa tan bajos en CliffWalking comparado con Q-Learning?**

Esto ocurre porque Monte Carlo espera a terminar el episodio para actualizar la tabla Q. En entornos con penalizaciones por paso y caídas, el agente acumula una cantidad masiva de recompensas negativas antes de realizar su primera actualización. A diferencia de Q-Learning, que aprende paso a paso mediante diferencias temporales, Monte Carlo sufre de una varianza extrema que "envenena" sus estimaciones de valor si no alcanza la meta rápidamente.

### **Prompt 4. En FrozenLake estocástico, ¿qué diferencia práctica hay entre la actualización de SARSA y Q-Learning?**

La diferencia reside en cómo gestionan el riesgo del "hielo resbaladizo". SARSA es on-policy y actualiza el valor considerando la acción que realmente va a tomar (que puede ser un error de exploración), lo que le lleva a aprender rutas más seguras. Q-Learning es off-policy y actualiza asumiendo que siempre tomará la mejor acción posible en el siguiente estado, lo que le permite encontrar la ruta óptima pero le hace caer más veces durante el entrenamiento debido a su naturaleza arriesgada.

### **Prompt 5. ¿Por qué mi agente DQN tiene "caídas" repentinas en su rendimiento después de haber alcanzado la recompensa máxima?**

Este fenómeno se conoce como "olvido catastrófico" o inestabilidad por sobreajuste. En entornos como CartPole, cuando el agente se vuelve muy bueno, el Replay Buffer se llena casi exclusivamente de experiencias de "éxito" (poste equilibrado). La red neuronal deja de ver ejemplos de fallos y "olvida" cómo reaccionar ante ángulos extremos. Para evitarlo, se recomienda usar un Replay Buffer más grande, ajustar el decaimiento de epsilon para mantener una exploración residual o emplear técnicas como Prioritized Experience Replay.

## 7. Bibliografía y referencias

- **Profesor/a [Hernández Molinero], [LD]. (2026).** Material de la asignatura Extensiones de Machine Learning. Máster en Inteligencia Artificial, Universidad de Murcia.
- **A. Maroti, "Reward Based Epsilon Decay," GitHub Pages, [En línea].** Disponible en: <https://aakash94.github.io/Reward-Based-Epsilon-Decay/>
- **Hernández Molinero, L. D. (2026).** Aprendizaje por Refuerzo: Procesos de Decisión de Markov. Material de la asignatura Extensiones de Machine Learning. Máster en Inteligencia Artificial, Universidad de Murcia.
- **Sutton, R. S., & Barto, A. G. (2018).** Reinforcement Learning: An Introduction (2nd ed.). MIT Press. Referencia fundamental para la formulación de las ecuaciones de Bellman y la comparación entre métodos TD y Monte Carlo.
- **Silver, D. (2015).** Lectures on Reinforcement Learning. University College London. Recurso utilizado para profundizar en la teoría de funciones de valor y políticas óptimas.
- **Gymnasium Documentation (2024).** Training an Agent. Farama Foundation. Documentación técnica utilizada para la implementación de los entornos FrozenLake y CliffWalking.
- **DeepMind x UCL. (2018).** Reinforcement Learning Series. Impartido por Hado van Hasselt. Recurso audiovisual para el estudio de métodos aproximados y tabulares.
- **Mnih, V., et al. (2015).** Human-level control through deep reinforcement learning. Nature, 518(7540), 529-533. (Paper original de DeepMind sobre DQN que introduce el Replay Buffer y la Target Network).



- **François-Lavet, V., et al. (2018).** An Introduction to Deep Reinforcement Learning. Foundations and Trends in Machine Learning. (Recurso para profundizar en los problemas de inestabilidad y convergencia en métodos aproximados).
- Enlace a github del proyecto: [Aprendizaje por Refuerzo Miguel Pastor](#)